# Programovanie v C++: Správa pamäte (Memory Management)

## Prednáška 5: Dynamická alokácia, deštruktory a kopírovanie

Programovanie mechatronických aplikácií
ÚAMT FEI STU

PMEAP Prednáška 5

# Opakovanie: Konštruktory

Konštruktor je špeciálna metóda, ktorá sa automaticky volá pri vytvorení inštancie triedy.

```
1  class Integer {
2  public:
3      int val;
4      Integer() {
5          val = 0; cout << "default constructor" << endl;
6      }
7  };
8
9  int main() {
10     Integer i; // Vypise: default constructor
11 }
```

# Opakovanie: Konštruktory a polia objektov

Ak vytvárame pole objektov, **predvolený konštruktor (default constructor) sa vyvolá pre každý jeden prvok poľa**.

```
int main() {
    Integer arr[3];
}
/* Vystup:
default constructor
default constructor
default constructor
*/
```

# Vnorené objekty a konštruktory

Pri vytváraní inštancie triedy sa automaticky vyvolávajú predvolené konštruktory všetkých jej členských polí (fields).

```
1  class IntegerWrapper {
2  public:
3      Integer val;
4      IntegerWrapper() {
5          cout << "IntegerWrapper default constructor" << endl;
6      }
7  };
8
9  int main() {
10     IntegerWrapper q;
11 }
12 /* Vystup: najprv sa inicializuje 'val', potom samotny 'Wrapper'
13 Integer default constructor
14 IntegerWrapper default constructor
15 */
```

# Parametrizované konštruktory

Konštruktory môžu prijímať argumenty pre inicializáciu špecifických hodnôt.

```
1  class Integer {
2  public:
3      int val;
4      Integer(int v) {
5          val = v; cout << "constructor with arg " << v << endl;
6      }
7  };
8
9  int main() {
10     Integer i(3);
11     // Implicitne volanie jednoparametrickeho konstruktora cez priradenie:
12     Integer j = 5;
13 }
```

Priradenie hodnoty príslušného typu vyvolá konštruktor s jedným parametrom.

# Strata predvoleného konštruktora

Ak v triede explicitne definujete konštruktor s parametrami, kompilátor **prestane automaticky poskytovať predvolený konštruktor**.

```
1  class Integer {
2  public:
3      int val;
4      Integer(int v) { val = v; }
5  };
6
7  int main() {
8      Integer i(3); // OK
9      Integer j;    // ERROR: No default constructor available
10 }
```

# Dôsledky chýbajúceho predvoleného konštruktora

Bez predvoleného konštruktora nie je možné deklarovať polia bez ich okamžitej explicitnej inicializácie.

```
1 int main() {
2     // Toto je v poriadku - explicitna inicializacia:
3     Integer a[] = { Integer(2), Integer(5) };
4
5     // ERROR: Zlyha, pretoze neexistuje defaultny konstruktor:
6     Integer b[2];
7 }
```

# Riešenie: Viacnásobné konštruktory alebo defaultné argumenty

Problém môžeme riešiť definovaním samostatného konštruktora bez argumentov, alebo použitím predvolených argumentov (default arguments).

```
1  class Integer {
2  public:
3      int val;
4      // Parameter 'v' ma defaultnu hodnotu 0
5      Integer(int v = 0) {
6          val = v;
7      }
8  };
9
10 int main() {
11     Integer i;     // OK, pouzije v=0
12     Integer j(3); // OK, pouzije v=3
13 }
```

# Ukazovateľ 'this'

Ako v metóde referovať na pole (field), keď má argument metódy rovnaký názov? Využijeme ukazovateľ `this`, ktorý ukazuje na aktuálnu inštanciu.

```
1  class Integer {
2  public:
3      int val;
4      Integer(int val = 0) {
5          this->val = val;
6          // this->val je skratka pre (*this).val
7      }
8
9      void setVal(int val) {
10         this->val = val;
11     }
12 };
```

`this->val` jednoznačne identifikuje členskú premennú objektu.

# Rozsah platnosti (Scoping) a Pamäť

- Kedykoľvek deklarujeme novú premennú (napr. `int x;`), v pamäti sa pre ňu alokuje priestor.
- Kedy sa táto pamäť môže uvoľniť a znova použiť?
- Pamäť sa uvoľní, keď premenná **vystúpi z rozsahu platnosti (goes out of scope)**.

# Zánik premennej a visiace smerníky (Dangling Pointers)

Keď premenná vystúpi z rozsahu platnosti, pamäťový blok už negarantuje uchovanie jej pôvodnej hodnoty.

```
1 int main() {
2     int *p;
3     if (true) {
4         int x = 5;
5         p = &x;
6     } // 'x' tu zanika, jej pamat sa moze znova pouzit
7
8     cout << *p << endl; // ??? Nedefinovane spravanie
9 }
```

Ukazovateľ p sa stáva tzv. **visiacim smerníkom (dangling pointer)**, pretože ukazuje na pamäť, ktorej obsah je už nedefinovaný.

# Kritická inžinierska chyba: Návrat lokálnej adresy

Úloha: Implementujte funkciu, ktorá vráti smerník na pamäť obsahujúcu číslo 5. **Nesprávna implementácia:**

```
1 int* getPtrToFive() {
2     int x = 5;  // 'x' je alokovana v lok lnom scope funkcie
3     return &x;
4 } // Pri navrate z funkcie 'x' zanika!
```

Akonáhle sa funkcia `getPtrToFive()` ukončí, premenná `x` vyjde z rozsahu platnosti. Funkcia tak vráti visiaci smerník (dangling pointer).

# Operátor 'new': Dynamická alokácia

Existuje iný spôsob alokácie pamäte, pri ktorom pamäť zostáva alokovaná, až kým ju **manuálne neuvoľníte**.

```
1 int *x = new int;
```

- Operátor `new` vracia smerník na novovytvorený pamäťový blok.
- Typ parametra (`int`) určuje, koľko bytov pamäte sa má alokovať.

# Terminológia pamäťových segmentov: Stack vs. Heap

V operačných systémoch rozlišujeme dva hlavné pamäťové priestory pre dáta programu:

- Ak použijeme statickú deklaráciu `int x;`, alokácia prebieha v oblasti pamäte nazývanej **Zásobník (Stack)**. Dáta na zásobníku sú automaticky riadené podľa rozsahu platnosti (scopu).
- Ak použijeme dynamickú deklaráciu `new int;`, alokácia prebieha v oblasti pamäte nazývanej **Halda (Heap)**. Táto pamäť je trvalá až do explicitného uvoľnenia.

# Operátor 'delete': Uvoľnenie pamäte

Operátor `delete` slúži na dealokáciu (uvoľnenie) pamäte, ktorá bola predtým alokovaná operátorom `new`.

```
1 int *x = new int;
2
3 // ... pouzitie pamate ...
4
5 delete x; // Prijima smernik na pamatovu lokaciu
```

Tento operátor zabezpečuje, že operačný systém môže daný pamäťový blok na halde znova využiť.

# Správna implementácia návratu smerníka

Alokovaním pamäte cez `new` zabezpečíme, že pamäť na halde zostane platná aj po skončení funkcie.

```
1  int *getPtrToFive() {
2      int *x = new int; // Alokacia na halde
3      *x = 5;
4      return x;
5  }
6
7  int main() {
8      int *p = getPtrToFive();
9      cout << *p << endl; // Vypise 5
10     delete p; // Po ukonceni prace Pamat uvolnime!
11 }
```

Keď skončíte prácu s pamäťou, je nutné ju uvoľniť operátorom `delete`.

# Úniky pamäte (Memory Leaks)

Ak uvoľnenie pamäte zanedbáte, vaša aplikácia bude zbytočne plytvať pamäťou OS. Stav, kedy program alokuje pamäť, no nie je schopný ju uvoľniť, nazývame **Memory Leak (Únik pamäte)**.

```
1 int main() {
2     int *p;
3     for (int i = 0; i < 3; ++i) {
4         p = getPtrToFive(); // V KAZDEJ iteracii alokuje novy blok!
5         cout << *p << endl;
6     }
7 }
```

Nesprávny kód - Memory Leak

# Analýza úniku pamäte v cykle

Čo sa presne deje na Halde (Heap) počas tohto cyklu?

- **1. iterácia:** Smerník `p` ukazuje na novovytvorený blok s hodnotou 5.
- **2. iterácia:** Volá sa opäť `new`. Smerník `p` sa prepíše a ukazuje na *druhý* blok. Prvý blok je navždy stratený v pamäti (žiadny smerník naň neukazuje).
- **3. iterácia:** Volá sa opäť `new`. `p` ukazuje na *tretí* blok. Predchádzajúce dva bloky sú nedostupné a trvalo zaberajú RAM.

# Chybný pokus o opravu úniku pamäte

Pridanie príkazu `delete` až za cyklus tento únik pamäte neopraví.

```
1 int main() {
2     int *p;
3     for (int i = 0; i < 3; ++i) {
4         p = getPtrToFive();
5     }
6     delete p; // Uvolni IBA posledny alokovany blok!
7 }
```

Uvoľní sa totiž výlučne tá pamäť, ktorá bola alokovaná v poslednej iterácii (na ktorú p ukazuje). Ostatné bloky zostávajú stratené.

# Správne vyriešenie úniku pamäte

Pre správnu fixáciu úniku musíme pamäť dealokovať priamo **vo vnútri cyklu**, kým k nej ešte máme prístup cez smerník p.

```
1 int main() {
2     int *p;
3     for (int i = 0; i < 3; ++i) {
4         p = getPtrToFive();
5         cout << *p << endl;
6
7         delete p; // Korektne uvolnenie v tej istej iteracii
8     }
9 }
```

Takto sa v každej iterácii vytvorí blok a hneď sa po použití aj korektne uvoľní.

# Zásady: Nepoužívajte pamäť po jej uvoľnení (Use After Free)

**Nesprávne:**

```
1 int *x = getPtrToFive();
2 delete x;
3 cout << *x << endl; // ??? Nedefinovane spravanie!
```

Pokus o prístup k dereferencovanému ukazovateľu, ktorého pamäť už bola vrátená operačnému systému (cez delete), je závažná chyba.

**Správne:**

```
1 int *x = getPtrToFive();
2 cout << *x << endl; // Najprv pamat pouzite (vypis)
3 delete x;           // Uvolnite az na uplnom konci
```

Pristupujte k pamäti iba vtedy, keď je stále alokovaná.

# Zásady: Dvojité uvoľnenie (Double Free)

Ďalšou fatálnou chybou je pokus o odstránenie tej istej pamäte dvakrát.

```
1 int main() {
2     int *x = getPtrToFive();
3     cout << *x << endl;
4
5     delete x;
6     delete x; // ERROR: Program spadne (Double free corruption)
7 }
```

Na pamäťový blok je dovolené zavolať operátor `delete` presne jedenkrát.

# Zásady: Uvoľňujte iba dynamicky alokovanú pamäť

Na pamäť alokovanú na zásobníku (stack) nesmiete nikdy volať operátor `delete`.

```
1 int main() {
2     int x = 5;          // Alokovane automaticky na zasobniku
3     int *xPtr = &x;
4
5     cout << *xPtr << endl;
6
7     // ERROR: xPtr neukazuje na 'new' pamat, neda sa uvolnit!
8     delete xPtr;
9 }
```

Uvoľňujte iba tú pamäť, ktorá bola exaktne alokovaná pomocou `new`.

# Dynamické alokovanie polí: Obmedzenia zásobníka

Keď alokujete pole na zásobníku zápisom `int arr[SIZE]`, veľkosť poľa `SIZE` **musí byť statická konštanta** známa počas kompilácie.

```
int numItems;
cin >> numItems;
// ERROR v standardnom C++: Velkost zasobnikoveho pola
// nemoze byt zadefinovana premennou pocas behu!
int arr[numItems];
```

# Dynamické alokovanie polí: Operátor new[]

Ak použijeme na alokáciu poľa operátor `new[]`, môže mať pole **variabilnú veľkosť** určenú až počas behu programu (at runtime).

```
1 int numItems;
2 cout << "how many items?";
3 cin >> numItems;
4
5 // new [typ_prvkov][pocet_prvkov]
6 int *arr = new int[numItems];
```

- `int` určuje typ prvkov, ktoré budú v poli uložené.
- `numItems` je premenná určujúca celkový počet dynamicky alokovaných prvkov.

# Uvoľňovanie dynamických polí: delete[]

Na uvoľnenie poľa alokovaného cez `new[]` je **bezpodmienečne nutné použiť operátor** `delete[]`.

```
1 int *arr = new int[numItems];
2
3 // ... praca s polom arr ...
4
5 delete[] arr; // Korektne uvolnenie CELEHO pola
```

Použitie obyčajného `delete` by v tomto prípade viedlo k nedefinovanému správaniu (často by sa uvoľnil len prvý prvok poľa).

# Alokácia inštancií tried (Objektov)

Operátor `new` je samozrejme možné použiť aj na dynamickú alokáciu inštancie akejkoľvek používateľsky definovanej triedy.

```
1 class Point {
2 public:
3     int x, y;
4 };
5
6 int main() {
7     Point *p = new Point; // Alokacia objektu na halde
8     delete p;
9 }
```

# Triedy a automatické volanie konštruktorov

Pri alokácii cez `new` sa kompilátor postará o to, aby sa okamžite vyvolal príslušný konštruktor daného objektu.

```
class Point {
public:
    int x, y;
    Point(int nx, int ny) {
        x = nx; y = ny;
        cout << "2-arg constructor" << endl;
    }
};

int main() {
    // Alokacia s priamym predanim argumentov do konstruktora:
    Point *p = new Point(2, 4);
    delete p;
} // Vypise: 2-arg constructor
```

Dynamická inštanciácia plne podporuje parametrizované konštruktory.

# Deštruktory (Destructors)

Opakom konštruktora je **Deštruktor**. Ide o metódu, ktorá sa automaticky volá tesne pred tým, ako je inštancia triedy dealokovaná z pamäte.

```
1  class Point {
2  public:
3      int x, y;
4      Point() {
5          cout << "constructor invoked" << endl;
6      }
7
8      // Destruktor je definovany vlnovkou (tildou)
9      ~Point() {
10         cout << "destructor invoked" << endl;
11     }
12 };
```

# Kedy sa spúšťa deštruktor?

Čas vyvolania deštruktora závisí od toho, akým spôsobom bol objekt alokovaný:

- **Na halde (cez new):** Deštruktor sa vyvolá presne v okamihu, kedy nad daným smerníkom explicitne zavoláte `delete`.

```
1 Point *p = new Point; // Vypise: constructor invoked
2 delete p;             // Vypise: destructor invoked
```

# Kedy sa spúšťa deštruktor? (Zásobník)

- **Na zásobníku (Stack-allocated):** Deštruktor sa vyvolá automaticky vo chvíli, kedy objekt opustí svoj rozsah platnosti (goes out of scope).

```
1 int main() {
2     if (true) {
3         Point p; // Vypise: constructor invoked
4     } // Vypise: destructor invoked (tu 'p' opusta scope)
5
6     cout << "p out of scope" << endl;
7 }
```

# Praktický príklad: Trieda reprezentujúca pole

Keď v C++ pracujeme s poľom, zvyčajne si musíme spoločne udržiavať smerník na jeho prvý prvok a zároveň aj počet jeho prvkov (veľkosť). Združme tieto údaje do jednej triedy.

```
1 class IntegerArray {
2 public:
3     int *data;  // Smernik na prvy prvok pola
4     int size;   // Pocet prvkov v poli
5 };
```

Smerník `data` obsluhuje dynamické pole prvkov na halde, zatiaľ čo `size` udržuje ich kvantitu.

# Manuálna obsluha poľa z vonku

Pri jednoduchej štruktúre musíme všetku pamäťovú agendu riešiť manuálne v klientskom kóde:

```
1  int main() {
2      IntegerArray arr;
3      arr.size = 2;
4      arr.data = new int[arr.size]; // Manualna alokacia
5
6      arr.data[0] = 4;
7      arr.data[1] = 5;
8
9      delete[] arr.data; // Manualna dealokacia!
10 }
```

Tento postup je prácny a nebezpečný. Logiku alokácie **presunieme do konštruktora**.

# Zapuzdrenie alokácie a dealokácie

Alokáciu vložíme do konštruktora a dealokáciu (`delete[]`) skryjeme do deštruktora. Takto zabezpečíme automatické riadenie pamäte.

```
1  class IntegerArray {
2  public:
3      int *data; int size;
4
5      IntegerArray(int size) {
6          data = new int[size]; // Alokacia podla poziadavky
7          this->size = size;
8      }
9
10     ~IntegerArray() {
11         delete[] data; // Automaticke uvolnenie pamate
12     }
13 };
```

# Použitie autonómnej triedy

Užívateľský kód sa radikálne zjednoduší a vyhne sa Memory Leakom:

```
int main() {
    // Alokuje pole pre 2 prvky priamo vo svojom vnutri
    IntegerArray arr(2);

    arr.data[0] = 4;
    arr.data[1] = 5;

    // ZIADEN 'delete[]' V MAIN METODE NIE JE POTREBNY!
} // Pri zaniku objektu 'arr' sa automaticky zavola jeho destruktor.
```

# Nebezpečenstvo predvoleného (plytkého) kopírovania

Čo sa ale stane, ak takýto objekt s vnútornou správou dynamickej pamäte prekopírujeme? Zoberme si nasledujúci chybný kód:

```
1  int main() {
2      IntegerArray a(2);
3      a.data[0] = 4; a.data[1] = 2;
4
5      if (true) {
6          IntegerArray b = a; // Predvoleny 'copy' konstruktor!
7      } // Objekt 'b' tu zanika a vyvolava sa JEHO destruktor
8
9      cout << a.data[0] << endl; // ??? Program zlyhava / not 4!
10 }
```

# Analýza chyby plytkého kopírovania (Shallow Copy)

- **Krok 1:** Defaultný kopírovací konštruktor (`IntegerArray b = a`) len otrocky skopíruje všetky polia (adresa poľa `a.data` sa prekopíruje do `b.data`). Oba objekty ukazujú na **rovnaký fyzický pamäťový blok**.
- **Krok 2:** Keď objekt `b` na konci bloku `if` vystúpi z platnosti (goes out of scope), vyvolá sa jeho deštruktor. Deštruktor `b` vykoná `delete[] data`, čím zničí pole na halde.
- **Krok 3:** Smerník `a.data` v objekte `a` sa týmto stáva visiacim smerníkom (dangling pointer).
- **Krok 4:** Keď potom opúšťa scope aj objekt `a`, jeho deštruktor sa pokúsi zavolať `delete[]` na už predtým zmazanom bloku pamäte. Vzniká kritická chyba **Double Free** a program havaruje.

# Vlastný Copy Constructor (Hlboká kópia)

Tieto chyby vyriešime napísaním vlastného kopírovacieho konštruktora (*Copy Constructor*), ktorý vykoná tzv. hlbokú kópiu (Deep Copy).

```
IntegerArray(IntegerArray &o) {
    // Alokujeme UPLNE NOVE pole pre tento novy objekt
    data = new int[o.size];
    size = o.size;

    // Rucne prekopirujeme hodnoty z povodneho objektu (o.data)
    for (int i = 0; i < size; ++i) {
        data[i] = o.data[i];
    }
}
```

# Bezpečnosť hlbokej kópie

S naším vlastným kopírovacím konštruktorom:

- **IntegerArray b = a;** vyvolá náš konštruktor. Objekt `b` dostane svoje vlastné, fyzicky oddelené pole.
- Dáta `{4, 2}` z poľa objektu `a` sa do poľa objektu `b` iba duplikujú.
- Keď objekt `b` opustí scope, zmaže iba svoje vlastné pole. Objekt `a` a jeho pole `a.data` zostávajú plne platné a nedotknuté.
- Výpis `a.data[0]` na konci bezpečne vypíše hodnotu `4`.

# Q & A

Ďakujem za pozornosť!